Got it 👍 — let's **reset and do Day 1 cleanly and correctly**, aligned with your final architecture (no Streamlit, proper modules, future-proof).

This is your **exact Day 1 execution** — no confusion, no extra theory.

✅ 🗓️ DAY 1 — PROJECT SETUP + CODE SPLIT (FINAL VERSION)

🎯 Goal

- Convert your current script → **modular system**
- Set foundation for **multimodal + RAG + agents**
- System should **run successfully**

📦 1. CREATE PROJECT

mkdir multimodal_llm

cd multimodal_llm

📁 2. CREATE EXACT FOLDER STRUCTURE

mkdir app ingestion embeddings retrieval tools models memory data

``

Now create files:

touch app/main.py

touch app/router.py

touch app/agent.py

touch ingestion/loader.py

touch ingestion/cleaning.py

touch ingestion/chunking.py

touch embeddings/embedding_service.py

touch embeddings/vector_store.py

touch retrieval/bm25.py

touch retrieval/hybrid.py

touch retrieval/reranker.py

touch tools/excel_tool.py

touch tools/pdf_tool.py

touch tools/image_tool.py

touch models/llm.py

touch memory/chat_memory.py

touch config.py

touch requirements.txt

🧠 3. WHAT YOU ARE DOING TODAY

👉 NOT building features
👉 ONLY:

- splitting your old code
- making system runnable
- preparing architecture

🧩 4. COPY YOUR EXISTING CODE (EXACT MAPPING)

✅ **4.1 Vector Store**

📁 embeddings/vector_store.py

```
import chromadb


class VectorStore:
    def __init__(self, path="./data/db", collection_name="docs"):
        self.client = chromadb.PersistentClient(path=path)
        self.collection = self.client.get_or_create_collection(name=collection_name)

    def add(self, ids, documents, embeddings, metadatas):
        self.collection.add(
            ids=ids,
            documents=documents,
            embeddings=embeddings,
            metadatas=metadatas
        )

    def query(self, embedding, top_k=5):
        return self.collection.query(
            query_embeddings=[embedding],
            n_results=top_k
        )
```

✅ **4.2 BM25 + Fusion**

📁 retrieval/bm25.py

```
from rank_bm25 import BM25Okapi


class BM25Retriever:
    def init(self):
        self.bm25 = None
        self.docs = []


    def fit(self, documents):
        self.docs = documents
        tokenized = [doc.split() for doc in documents]
        self.bm25 = BM25Okapi(tokenized)


    def search(self, query, top_k=5):
        scores = self.bm25.get_scores(query.split())
        ranked = sorted(
            zip(self.docs, scores),
            key=lambda x: x[1],
            reverse=True
        )
        return [doc for doc, _ in ranked[:top_k]]
```

📁 retrieval/hybrid.py

```
def reciprocal_rank_fusion(bm25_docs, vector_docs, k=60):
    scores = {}


    for i, doc in enumerate(bm25_docs):
```

```
            scores[doc] = scores.get(doc, 0) + 1 / (i + k)

    for i, doc in enumerate(vector_docs):
        scores[doc] = scores.get(doc, 0) + 1 / (i + k)

    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

✅ 4.3 Chat Memory

📁 memory/chat_memory.py

```
from collections import deque

class ChatMemory:
    def __init__(self, size=10):
        self.memory = deque(maxlen=size)

    def add(self, user, response):
        self.memory.append({
            "user": user,
            "response": response
        })

    def get(self):
        return list(self.memory)
```

✅ 4.4 LLM (Qwen via Ollama)

📁 models/llm.py

```
import ollama


def generate(prompt):
    response = ollama.chat(
        model="qwen:latest",
        messages=[{"role": "user", "content": prompt}]
    )
    return response["message"]["content"]
```

✅ **4.5 Cleaning**

📁 ingestion/cleaning.py

```
import re


def clean_text(text):
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"\n+", " ", text)
    return text.strip()
```

✅ **4.6 Chunking**

📁 ingestion/chunking.py

```
def chunk_text(text, size=600, overlap=100):
    chunks = []
```

```
    start = 0

    while start < len(text):
        chunks.append(text[start:start + size])
        start += size - overlap

    return chunks
```

✅ **4.7 Minimal Agent (Don’t Overthink)**

📁 app/agent.py

```
from models.llm import generate

def run_agent(query):
    return generate(query)
```

✅ **4.8 Router**

📁 app/router.py

```
def route(query):
    q = query.lower()

    if "excel" in q:
        return "excel"
    elif "image" in q:
        return "image"
    else:
        return "rag"
```

✅ **4.9 Main Entry**

📁 app/main.py

```
from app.router import route
from app.agent import run_agent


def main():
    print("Multimodal AI System Started")

    while True:
        query = input(">> ")

        if query.lower() == "exit":
            break

        task = route(query)

        if task == "rag":
            response = run_agent(query)
        else:
            response = f"{task} module not implemented yet"

        print(response)

if __name__ == "__main__":
    main()
```

📦 5. REQUIREMENTS

📁 requirements.txt

chromadb

rank-bm25

sentence-transformers

pandas

ollama

▶️ 6. RUN SYSTEM

python app/main.py

Expected:

Multimodal AI System Started

>>

✅ DAY 1 FINAL CHECKLIST

You are ✅ ONLY if:

- ✅ Folder structure complete
- ✅ Code split into proper modules
- ✅ No errors on run
- ✅ Qwen responds
- ✅ No feature added yet

⚠️ DO NOT DO TODAY

- ❌ embeddings integration
- ❌ PDF ingestion
- ❌ Excel tool

❌ OCR
❌ agent improvements

🧠 **WHAT YOU BUILT TODAY**

👉 Not a script anymore
👉 You now have:

✅ **Enterprise AI system skeleton**

🚀 **NEXT**

👉 Say **“Day 2”**
I’ll give you:

- embedding_service.py (real implementation)
- connect with vector store
- first working RAG query
